

Module 10 — Integration Task: Multi-Service Docker Compose Stack (Team)

Due: Thursday end of day. Resubmissions are accepted through Saturday of the assignment week.

This is a team deliverable. Integration 10 is delivered by instructional-team-assigned teams of 3 learners — your instructional team forms the teams (you do not self-select) and assigns each Team Member to one of three role surfaces: Backend lead, Frontend lead, or Infra-Integration lead. Your team leader and per-Team-Member role assignment are communicated by the instructional team via a pinned cohort Slack message by Wednesday AM.

You will compose the Lab's FastAPI backend and Next.js frontend with **containerized Neo4j** and **containerized Weaviate** into a one-command four-service Docker Compose stack. The team deliverable is `docker-compose-up -d` that brings the entire stack to a healthy state, populates the recipe graph from a seed Cypher script, populates the Weaviate vector index from a seed-and-embed script, and serves an end-to-end demo request from the browser through the Next.js page to the RAG (Retrieval-Augmented Generation) endpoint that returns a grounded cited answer.

Team formation, size, and role assignment

- Team formation method:** instructional-team-assigned. The instructional team uses live signal from Modules 1–9 grading to balance teams across skill levels.
- Team size:** 3 learners recommended. If cohort headcount does not divide evenly, the instructional team forms one or two teams of 2 (Frontend and Infra-Integration roles merge) or 4 (Infra-Integration splits into 'Compose + healthchecks' and 'Seed + runbook' surfaces).
- Role assignment:** roles are assigned by the instructional team. You do not self-select your role. Assignments are based on Lab signal (Backend lead → `Lab /rag/answer`; clear, Frontend lead → `Lab Next.js` pages; clear, Infra-Integration lead → `Lab Dockerfile` clear) and deliberate stretch (a Team Member who struggled with Next.js may be assigned Frontend lead with TA (Teaching Assistant) support).
- Roster comms:** by Wednesday AM of the instruction week, the instructional team posts a pinned cohort Slack message naming each team's three Team Members and their assigned roles, plus the team's Slack channel.

Setup

If you have not already installed the Module 10 packages, go back to the [Setup: install before the concept session](#) section of the reading and complete the install before proceeding.

Integration template repo: `levelup-ai/apl10-i10-110`. The designated team submitter clicks "Use this template" → "Create a new repository" on the template page (see "One team fork" below).

One team fork — one team submitter. A designated Team Member (typically the Infra-Integration lead, but the team chooses) clicks "Use this template" on `i10-i110` once to create the team's working repo (e.g., `i10-i10-team-[N]` under the team's chosen account or organization). The team submitter adds the other two Team Members as collaborators with Write access on the team fork.

```
# After the team submitter creates the team fork:
git clone https://github.com/cteam-fork-owner/i10-i10-team-N.git
cd i10-i10-team-N
git checkout -b backend/rag-endpoints # If you are the Backend lead
# or: git checkout -b frontend/nextjs-pages # Frontend lead
# or: git checkout -b infra/docker-compose # Infra-Integration lead
```

The starter ship:

- A pre-implemented copy of the Lab's `api/` backend (so a learner who did not finish the Lab can still attempt the integration).
- A pre-implemented copy of the Lab's `web/` Next.js frontend.
- The pre-built recipe `seed_cypher` and `seed_chunks.json`, plus the pre-implemented `seed_weaviate.py`.
- A `docker-compose.yml` skeleton with the four services; keys declared but body fields **TODOLARGE**.
- Empty stubs for `.env.example`, `scripts/seed_neo4j.sh`, `scripts/seed_weaviate.sh`, `scripts/healthcheck_stack.sh`, `README.md`.
- A pre-populated `TEAM.md` template (roster + per-role file checklist + escalation checklist).
- A pre-populated `CONTRIBUTING.md` (per-branch + internal-PR convention, role-to-reviewer mapping, contact-change protocol).

Team workflow — the shape that matters

The integration uses the **fork PR collaboration** pattern. The team fork has three role-named branches; each branch opens an **internal PR** against the team fork's `main`; the assigned teammate-reviewer approves before merge; once all three role branches merge, the team submitter opens the team submission.

Per-role branches

- Backend lead** → `backend/api-endpoints`
- Frontend lead** → `frontend/nextjs-pages`
- Infra-Integration lead** → `infra/docker-compose`

Branch names are recorded in `TEAM.md`. Do not rename them — the autograder and TA cross-reference scripts look for these exact names.

Internal PRs and role-to-reviewer mapping

Each role's branch opens an internal PR against the team fork's `main`. The reviewer is the Team Member whose role surface the change touches:

- Backend lead's `api/models.py` (OpenAPI shape change) → reviewed by **Frontend lead**.
- Frontend lead's `web/lib/types.ts` → reviewed by **Backend lead**.
- Infra-Integration lead's `docker-compose.yml` → reviewed by **Backend lead**.

At least one teammate approval is required before merging to the team fork's `main`. The autograder workflow does not run on internal PRs — it runs on the team submission PR. Rubber-stamping the internal review defeats the purpose: the TA spot-checks the internal-PR review comments during grading.

Contract-change protocol

If you (Backend lead) rename a Pydantic field, **announce it on the team Slack channel before the change lands**. The Frontend lead must update `lib/types.ts` in the same review cycle. Silent OpenAPI shape changes break the frontend after merge and the failure surfaces only at the team-submission Playwright stage.

If you (Infra-Integration lead) change `NEXT_PUBLIC_API_URL`, announce it. The Frontend lead's Next.js production build bakes that value at build time — changing it after the build does nothing until the next rebuild.

If you (Frontend lead) need a new backend field, open an internal PR comment on the Backend lead's branch, do not assume.

Escalation

Disagreements about scope, role boundaries, or contract changes escalate in this order:

- Inline comment on the internal PR.
- Team Slack channel with the TA tagged.
- Support Instructor (if the TA decision is contested).
- Lead Instructor (only if a role-rebalancing decision is needed).

Document the escalation path in `TEAM.md`. The pre-baked template ships with this checklist; fill it in as you go.

Per-role deliverables

Backend lead — owns the FastAPI service

Scope: extension and hardening, not authoring from scratch. Integration ships with `api/` already pre-implemented: the five path operations (`/extract`, `/kg/query`, `/rag/answer`, `/healthz`, `/readyz`). Pydantic shapes, lifespan-loaded Neo4j + Weaviate + spaCy + generator, the W9B mapper, the RAG composer, and the Dockerfile are all in place. The Lab autograder passes against this baseline. Your job is to **own, understand, and extend** this surface for the Integration deliverable — the team's `docker-compose-up demo`, the cross-boundary contract with the Frontend lead, and any production-hardening the integration rubric calls out.

Files you own (extend; do not rewrite from scratch) on `backend/api-endpoints`:

- `api/main.py` — keep the OpenAPI surface stable; wire Integration-specific service-name DNS (Domain Name System) env reads, error-response consistency across the three endpoints.
- `api/models.py` — extend shapes only when the Frontend lead has requested a field. No unilateral renames — `chunk_id` → `chunkid` is exactly the silent-drift trap the team contract exists to prevent.
- `api/rag.py` — integration details (per-call timeouts on the generator wrapper, structured logging the Infra-Integration lead can read) **without** weakening the grudging refusal that returns the sentinel when citations cannot be resolved.
- `api/deps.py` — extend `Depends()` chains as new endpoints or middleware are added.
- `api/dockerfile` — refine the cache layering and `.dockerignore`; do not rebase the package-relative-import discipline (CMD must remain `uvicorn api.main:app` from repo root).

You hold the **OpenAPI contract**. Every shape change is announced on the team Slack channel and reflected in the OpenAPI schema before the Frontend lead consumes it. **Removing** a field is a breaking change for the frontend even when the test suite still passes.

Frontend lead — owns the Next.js application

Files you author or modify on `frontend/nextjs-pages`:

- `web/pages/extract.tsx`, `web/pages/kg.tsx`, `web/pages/rag.tsx` — typed `fetch` calls; render typed results; handle 422 / 503 / network errors; render `[N]`-style citation markers on `/rag`.
- `web/lib/types.ts` — three TypeScript interfaces mirroring the Backend lead's Pydantic shapes exactly (field for field).
- `web/Dockerfile` — **pre-implemented and deliberately diverges from the Lab**. The Lab taught `node:20-alpine` + Next.js standalone output (image ~150 MB). The integration ships `node:20-slim` + the non-standalone `copy-node_modules` pattern instead — Alpine's `musl` lib cannot load the `libc/arc4-linux-x86_64-gnu` binary pinned in `packages/kg.json` (the alternate `libc/arc4-linux-x86_64-musl` binary is not present in lockfiles generated on macOS / Debian hosts, so the alpine build is brittle across team members). The slim image lands ~640 MB — above the Lab's < 400 MB target — and the comment at the top of the Dockerfile records this rationale. You may extend the Dockerfile (build args, caching layers) but the base image and standalone-vs-non-standalone choice are intentional: do not "fix" them back to the Lab pattern.
- `tests/frontend/playwright/extract/kg_xapi.spec.ts` — one Playwright smoke test per page.

You hold the **browser-side contract**. Any backend field rename you need is requested via an internal PR comment on the Backend lead's branch, not assumed.

Infra-Integration lead — owns the topology

Files you author or modify on `infra/docker-compose`:

- `docker-compose.yml` — the four services (`api`, `web`, `neo4j`, `weaviate`); Neo4j memory caps, healthchecks on both data tiers; named volumes (`neo4j_data`, `weaviate_data`); `depends_on` readiness-ordering chain; env wiring with the host-vs-container `NEXT_PUBLIC_API_URL` distinction.
- `.env.example` — placeholders for `NEO4J_AUTH`, `NEO4J_PASSWORD`, `WEB_ORIGIN`. The real `.env` is `.gitignore'd`.
- `scripts/seed_neo4j.sh`, `scripts/seed_weaviate.sh`, `scripts/healthcheck_stack.sh` — idempotent driver scripts.
- `README.md` runbook — end-to-end onboarding from clone to browser demo.
- `tests/integration/test_stack_e2e.py` — end-to-end smoke harness that brings the stack up and verifies the demo curl.

You hold the **stack contract** — `NEO4J_URL`, `WEAVIATE_URL`, `NEXT_PUBLIC_API_URL`, and the host-vs-container distinction are documented in the runbook and reviewed with both other leads.

API build-context discipline. The pre-implemented `api/` uses package-relative imports and its Dockerfile runs `uvicorn api.main:app` from `/app`. The `api` service in `docker-compose.yml` must use the repo root as the build context — `build: /api` will appear to work during `docker compose build` but the container will crash with `ModuleNotFoundError: No module named 'api'` at startup.

Team integration tasks (Infra-Integration lead authors; reviewed by Backend)

The four-service Compose stack (sections from the reading apply — see Sections 11–14).

1. `docker-compose.yml` — four services

Declare `api`, `web`, `neo4j`, `weaviate` as services. Declare `neo4j_data` and `weaviate_data` as named volumes.

2. Neo4j service

- Image `neo4j:5-community`.
- Port mapping `7687:7687` and `7474:7474`.
- Memory caps via `env`. `NEO4J JVM_MEMORY_HEAP_MAX_SIZE=1G` and `NEO4J JVM_MEMORY_PAGECACHE_SIZE=512M`.
- Named volume `neo4j_data` mounted at `/data`.
- Healthcheck via `cypher-shell -u $NEO4J_USER -p $NEO4J_PASSWORD 'RETURN 1'` (Compose-time interpolation from the host's `.env`; **do not use `$@` (double dollar)** — that defers expansion to the container shell, where `NEO4J_USER` and `NEO4J_PASSWORD` are not exported and the healthcheck silently authenticates as the empty user).

3. Weaviate service

- Image `semitechnologies/weaviate:1.24.10`.
- Port mapping `8080:8080`.
- `Env` `QUERY_DEFAULTS_LIMIT=20`, `AUTHENTICATION_ANONYMOUS_ACCESS_ENABLED=true`, `PERSISTENCE_DATA_PATH=/var/lib/weaviate`, `DEFAULT_VECTORIZER_MODULE=none` (we embed externally with sentence-transformers).
- Named volume `weaviate_data` mounted at `/var/lib/weaviate`.
- Healthcheck via `wget --spider --tries=1 http://localhost:8080/v1/.well-known/ready` — the `semitechnologies/weaviate` image ships with `wget`, not `curl`, so a `curl`-based healthcheck fails silently and Weaviate never reaches `healthy`.

4. API service

- Built from `/api`.
- `depends_on` (`api` | `neo4j` | `condition: service_healthy` |), `weaviate` (`condition: service_healthy` |) |).
- `Env` `NEO4J_URL=http://$NEO4J_USER:$NEO4J_PASSWORD@localhost:7687`, `WEAVIATE_URL=http://weaviate:8080`, `NEO4J_USERS=$NEO4J_USER`, `NEO4J_PASSWORD=$NEO4J_PASSWORD`, `WEB_ORIGIN=http://localhost:3000`.
- Port mapping `8080:8080`.
- Healthcheck via Python's `subprocess` `utils` `http` against `http://localhost:8080/ready`, with `start_period: 180s` so a cold HuggingFace cache (which downloads `spaCy` + `flan-t5-base` on first boot, up to ~4 min) does not exhaust the retries budget. (The `python:3.11-slim` base image does not ship `curl` — see the Reading's Section 11 (healthcheck note.) The Weaviate healthcheck uses `wget` (the image lacks `curl`); the Neo4j healthcheck uses `cypher-shell` (which ships in the `neo4j` image).

5. Web service

- Built from `/web` (multi-stage Node Dockerfile).
- `depends_on` (`api` | `condition: service_healthy` |) |).
- Built arg `NEXT_PUBLIC_API_URL=http://localhost:8080` (post-facto, baked at build time — the browser cannot resolve `api`).
- Port mapping `3000:3000`.
- Healthcheck via Node's `stdlib` `http` against `http://localhost:3000` (the `node:20-slim` base image ships neither `curl` nor `wget`): `[ "CMD", "node", "-e", "require('http').get('http://localhost:3000', r => process.exit(r.statusCode > 500 ? 0 : 1)).on('error', () => process.exit(1))" ]`. The CI workflow's Phase 3 wait requires `web` to report `healthy`; without this declaration, `web` stays "running" forever and the workflow times out.

6. `.env` discipline

- `.env.example` with placeholders for `NEO4J_AUTH=neo4j:password`, `NEO4J_USER=neo4j`, `NEO4J_PASSWORD=neo4j:password`, `WEB_ORIGIN=http://localhost:3000`. `NEO4J_USER` is read by the `api` service env and by the `seed/healthcheck` `cypher-shell` invocations — keeping it in `.env` instead of hardcoding `neo4j` across three places means a future Neo4j multi-tenant setup or auth rotation is a one-line change.
- `.gitignore` excludes `.env`. The real `.env` never ships.

7. Seed scripts

- `scripts/seed_neo4j.sh` pipes `seed_cypher` into the running Neo4j container via `docker compose exec -T neo4j cypher-shell -u $NEO4J_USER -p $NEO4J_PASSWORD`.
- `scripts/seed_weaviate.py` runs the seed **inside the api container** (`docker compose exec -T api python seed_weaviate.py`) — sentence-transformers, weaviate-client, and the rest of the `api` requirements live in the `api` image, not on the host. Running `python seed_weaviate.py` from the host `venv` works only if the learner has manually installed the `api` requirements locally; the script should not depend on that. Document expected runtime (~10–45 s depending on whether embeddings are pre-baked).
- Run both scripts from the **repo root** (the directory containing `docker-compose.yml`) with the stack already up — `docker compose exec` resolves the service name against the Compose project in the current working directory. Running them from inside `scripts/` produces `no configuration file provided: not found`.
- Both scripts are **idempotent** — re-running does not duplicate state. Use `MERGE` in Cypher; check Weaviate class existence before creating.

8. Runbook

The `README.md` walks a new engineer end-to-end:

- `git clone`
- `cp .env.example .env` and fill in `NEO4J_PASSWORD`.
- `docker compose up -d --build`
- Wait for all four services to report healthy (`docker compose ps`).
- bash `scripts/seed_neo4j.sh`
- bash `scripts/seed_weaviate.sh`
- Open `http://localhost:3000/rag` in a browser.
- Submit the seeded question "Find Sichuan recipes that use ginger" and observe a cited answer.
- `docker compose down -v` to teardown.

The runbook is graded — `commands work as documented` is a rubric dimension.

Tests — how the autograder validates the stack

The autograder is split across two CI (Continuous Integration) jobs. A **structural** job runs `pytest tests/` against the static repo artifacts (Compose topology, env discipline, `TEAM.CONTRIBUTING` presence, Playwright spec authoring, OpenAPI/types boundary). A **stack** job then brings the four-service stack up and exercises the end-to-end demo via shell + `curl` — **not `pytest`**. The runtime end-to-end harness at `tests/integration/test_stack_e2e.py` is a TODO scaffold that the Infra-Integration lead may author for local demo-prep; the autograder does not run it.

The **stack** job in CI on the team submission PRs:

- `docker compose up -d --build` — brings the four services up.
- Polls `docker compose ps` — `format json` until each tier reports "healthy" within its budget — data tier (Neo4j + Weaviate) ≤ 2 min, `api` ≤ 5 min (the `api` lifespan loads `spaCy` + `flan-t5-base`; cold HuggingFace caches add several minutes on a fresh runner), `web` ≤ 2 min.
- Verifies **readiness ordering** — Neo4j and Weaviate must reach healthy before the API; the API before the web. If a later-tier service goes healthy first, the workflow fails with the offending service name.
- Runs `seed_neo4j.sh` and `seed_weaviate.sh`.
- Issues `curl -X POST http://localhost:8080/rag/answer -H 'Content-Type: application/json' -d '{"question": "How do I prep ginger for stir-fry?"}'` and verifies HTTP 200 with `len(content) > 0`, `confidence > 0`, `answer` not equal to the empty-retrieval sentinel.
- Runs Playwright headless — navigates to `http://localhost:3000/rag`, types the seeded question, clicks submit, asserts the rendered answer text plus at least one inline citation marker.
- Idempotency check — `docker compose down -v` followed by `docker compose up -d` produces an empty-volume stack; re-running the seeds repopulates without error.
- Boundary checks — OpenAPI field names match `lib/types.ts` interface field names; `web` container can reach `api/healthz` via Compose service-name DNS; `seed_weaviate.py` is idempotent (second run does not duplicate chunks).
- Team artifacts — `TEAM.md` has all three role rows filled in; `CONTRIBUTING.md` ships with the internal-PR rule.

Run locally before submission:

```
# As Infra-Integration lead → full smoke
docker compose down -v
docker compose up -d --build
sleep 90
./seed_neo4j.sh
./seed_weaviate.sh
curl -X POST http://localhost:8080/rag/answer \
-H 'Content-Type: application/json' \
-d '{"question": "How do I prep ginger for stir-fry?"}'
```

Common pitfalls

Team-level integration pitfalls

- Contract changes land silently.** Backend lead merges an OpenAPI change without telling the Frontend lead. TypeScript does not know about the backend; the build still passes; Playwright fails at the team-submission stage. Mitigation: the `CONTRIBUTING.md` rule that shape changes require a Slack announcement plus a same-cycle Frontend update.
- Internal PR review is rubber-stamped.** The assigned reviewer approves without reading; the integration bug ships to the team submission PR. Mitigation: TA spot-checks internal-PR review comments during grading.
- One Team Member's work blocks the team.** The Frontend lead has not finished `lib/types.ts` and the team submission PR cannot pass Playwright. Mitigation: TA-monitored mid-cycle check-in on Wednesday end of day; if one Team Member is blocked, the TA escalates to the Support Instructor for role rebalancing.
- Role drift.** Backend lead "helps" by writing parts of `docker-compose.yml`; the Infra-Integration lead's per-role grading score drops because the contribution log shows they authored less. Mitigation: `TEAM.md` documents role boundaries; the PR per-role contribution summary makes the boundary auditable.

Per-role pitfalls

Backend lead. OpenAPI shape changes without team-channel announcement; CORS misconfigured for the Compose-network origin (set `WEB_ORIGIN=http://localhost:3000`, never `http://web:3000`); grounding contract dropped under integration pressure (removing the citation-presence check to "make the test pass" flips the demo-call test red later).

Frontend lead. TypeScript interface drift (`chunkid` vs `chunk_id`), `NEXT_PUBLIC_API_URL` set to the Compose service name (must be `http://localhost:8080`); multi-stage Dockerfile copies `node_modules` from host because the `.dockerignore` is missing — image balloons, CI times out.

Infra-Integration lead. `healthz` endpoint (`http://localhost:7687` or `http://localhost:8080`) inside the API container (must be Compose service-name DNS — `http://neo4j:7687`), `http://weaviate:8080`, omitting healthchecks (`depends_on` without `condition: service_healthy` only waits for container start, not ready); Weaviate `DEFAULT_VECTORIZER_MODULE` left unset (Weaviate tries to vectorize internally, conflicts with externally-embedded chunks, inserts silently fail); forgetting Neo4j memory caps (OOM (Out Of Memory) on a 16 GB laptop); committing `.env`.

Challenge Extensions

Each role surface has its own challenge tier. A teammate's challenge work does not affect you; challenges are individual extensions on top of the team deliverable.

Backend lead

- Tier 1.** Async path operations — refactor `/extract`, `/kg/query`, `/rag/answer` to `async def`; switch Neo4j to `AsyncDriverDatabase` and Weaviate to its async client.
- Tier 2.** Streaming RAG — convert `/rag/answer` to Server-Sent Events emitting partial generated tokens as the `flan-t5-base` produces them. Expose a `/rag/answer/stream` endpoint alongside the non-streaming one. The Frontend lead may consume it as their own challenge.
- Tier 3.** Retrieving + multi-hop RAG — cross-encoder reranker on top-20 Weaviate results down to top-4; multi-hop retrieval pattern where first-pass citations re-query Weaviate if `confidence < 0.8`; ship a held-out-question evaluation table.

Frontend lead

- Tier 1.** OpenAPI-driven TypeScript client — replace `lib/types.ts` with codegen output from the FastAPI OpenAPI schema (`openapi-typescript`); three pages report the generated client.
- Tier 2.** Streaming citation rendering — consume the Backend lead's `/rag/answer/stream` SSE production with EventSource and render tokens + citations incrementally. Requires Backend Tier 2.
- Tier 3.** Product with GradeNext.js — convert pages to App Router with React Server Components; add suspense boundaries for retrieval-in-flight states; add `loading.tsx` and `error.tsx` per page; ship a Lighthouse report.

Infra-Integration lead

- Tier 1.** Multi-environment overlays — `docker-compose.dev.yml` (source-mount for hot reload) + `docker-compose.prod.yml` (Uvicorn workers, next start, no source mount, restart policy `unless-stopped`); overlay merge command documented.
- Tier 2.** Backup, restore, and idempotent seed — `backup.sh` (Cypher dump + Weaviate snapshot API) and `restore.sh` (idempotent seed scripts (`CREATE CONSTRAINT IF NOT EXISTS`), `MERGE`); Weaviate class-creation skip-if-exists; tests verify backup → teardown → restart → restore round-trip preserves counts.
- Tier 3.** Production deployment pipeline — GitHub Actions workflow that on PR merge to `main` builds the four images, tags them with git SHA, pushes to GitHub Container Registry, generates a pinned production Compose file, and ships a rollback runbook.

Submission

Team submission (one per team)

- All three role branches merge to the team fork's `main` via internal PRs (each with at least one teammate approval).
- Each Team Member confirms `docker compose up -d` works locally on their own machine.
- The team submitter opens one **PR from the team fork to upstream** `i10-i110` (or, equivalently, pastes the team fork's `main` branch URL into Talent.MS — your instructional team will confirm which submission mode is enabled for your cohort).
- The team submitter pastes the team submission URL into Talent.MS → Module 10 → Integration 10.

Your team submission PR description must include:

- `docker-compose` output showing all four services healthy.
- Output of `seed_neo4j.sh` and `seed_weaviate.sh`.
- The demo curl command on `/rag/answer` and its 200 response including citations.
- A screenshot of the Next.js `/rag` page rendering a cited answer to the seeded question.
- The `TEAM.md` roster (each Team Member named with their assigned role).
- Per-role contribution summary** — one paragraph per Team Member — naming the files each Team Member authored. The TA cross-references this against `git log --author=email` on the team fork and against the internal-PR review history. The pre-baked `TEAM.md` per-role file checklist is the reference key.
- Paste this URL into Talent.MS → Module 10 → Integration 10 to submit this assignment.

Per-Team-Member participation confirmation (one per Team Member)

Each Team Member separately submits a Talent.MS checkbox in Talent.MS → Module 10 → Integration 10 Participation Confirmation form:

- Their name and the assigned role (Backend lead / Frontend lead / Infra-Integration lead).
- The files they authored (matches the per-role contribution paragraph in the team PR description).

The participation confirmation drives the **per-role attribution tier (40 points)** of the Integration grading. Without it, the TA cannot attribute the per-role score and the Team Member's per-role tier defaults to the team tier score.

